API SECURITY RISK ANALYSIS REPORT

Prepared By: Keerthan CS
Track: Cyber Security (Future Interns)

---

1. Executive Summary

An API security risk analysis was conducted on sample application endpoints to identify vulnerabilities related to authentication, authorization, and data exposure. This report outlines the identified insecure API endpoints, their potential business impact, and actionable remediation steps to secure the application architecture.

2. Risk Finding 1: Broken Object Level Authorization (BOLA)

- **Vulnerability:** The API endpoint `/api/v1/users/{user_id}/financial_data` does not adequately validate if the requesting user has the authorization to access the specific `user_id` requested.
- **Risk Level:** High
- **Business Impact:** An attacker can change the `user_id` parameter in the API request to view or modify the private financial data of other customers. This leads to a massive breach of Personally Identifiable Information (PII) and severe regulatory fines.
- **Remediation:** Implement strict authorization checks on the server side. Ensure the API validates that the currently authenticated user has explicit permission to access the requested data object.

3. Risk Finding 2: Missing Rate Limiting and Brute-Force Protection

- **Vulnerability:** The authentication endpoint `/api/v1/login` allows unlimited sequential requests without locking the account, requiring a CAPTCHA, or throttling the connection.
- **Risk Level:** Medium
- **Business Impact:** Hackers can use automated tools to launch "credential stuffing" or brute-force attacks against the API, testing thousands of stolen passwords per minute until they gain access to legitimate user accounts.
- **Remediation:** Configure the API Gateway to enforce rate limiting (e.g., maximum 5 login attempts per minute per IP address). Implement account lockout mechanisms after consecutive failed attempts.

4. Risk Finding 3: Excessive Data Exposure

- **Vulnerability:** The endpoint `/api/v1/users/profile` returns the entire database object for a user (including hashed passwords, internal role IDs, and security reset answers) to the client application, relying on the front-end to filter out what the user sees.
- **Risk Level:** Medium
- **Business Impact:** Anyone inspecting the raw API response traffic using browser developer tools can view sensitive backend data that was never meant to be displayed on the screen.
- **Remediation:** Implement data minimization. The API backend must be configured to filter responses and only return the exact data fields required by the user interface.